159.102 Notes

Section 3 : Adder circuits + Real Numbers + bitwise operations

In this section we look at the design of the circuits required to carry out arithmetic in a computer. We also look at how real numbers are stored. Then we get back to C++ programming with bitwise operations.

Integer arithmetic in a computer

Add: we need to be able to add two binary numbers.

Subtract: `a - b` can be rewritten as `a + (-b)`. So a computer never needs to subtract. We can simply set up the bit storage pattern that represents the negative number by following the twos complement process and then we add.

Multiply: note that `3 * a = a + a + a`. Thus any multiplication is actually a series of additions.

Divide: division consists of a series of subtractions, possibly keeping track of remainders, etc. And a series of subtractions in a computer is actually a series of additions.

Any complex mathematical formula will eventually break down into a series of additions.

Thus the only operation required in a computer is addition which means that the only circuit that a computer needs (to do calculations) is the adder circuit.

The half-adder circuit

We need circuits to perform addition inside a computer.

Assume a word size of 6 bits and look at an addition, e.g. 5 + 1

```
  5 = 00 0101      Number A made up of 6 bits
+ 1 = 00 0001      Number B made up of 6 bits
      ---------
      00 0110      The answer is 6.
```

Now look at just one column of this addition. Each column has 2 inputs A and B (each part of the corresponding binary numbers A and B) and each column has 2 outputs Result and Carry-out.

We set up a truth table for every possibility as follows:

A	B	Result	Carry-out
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

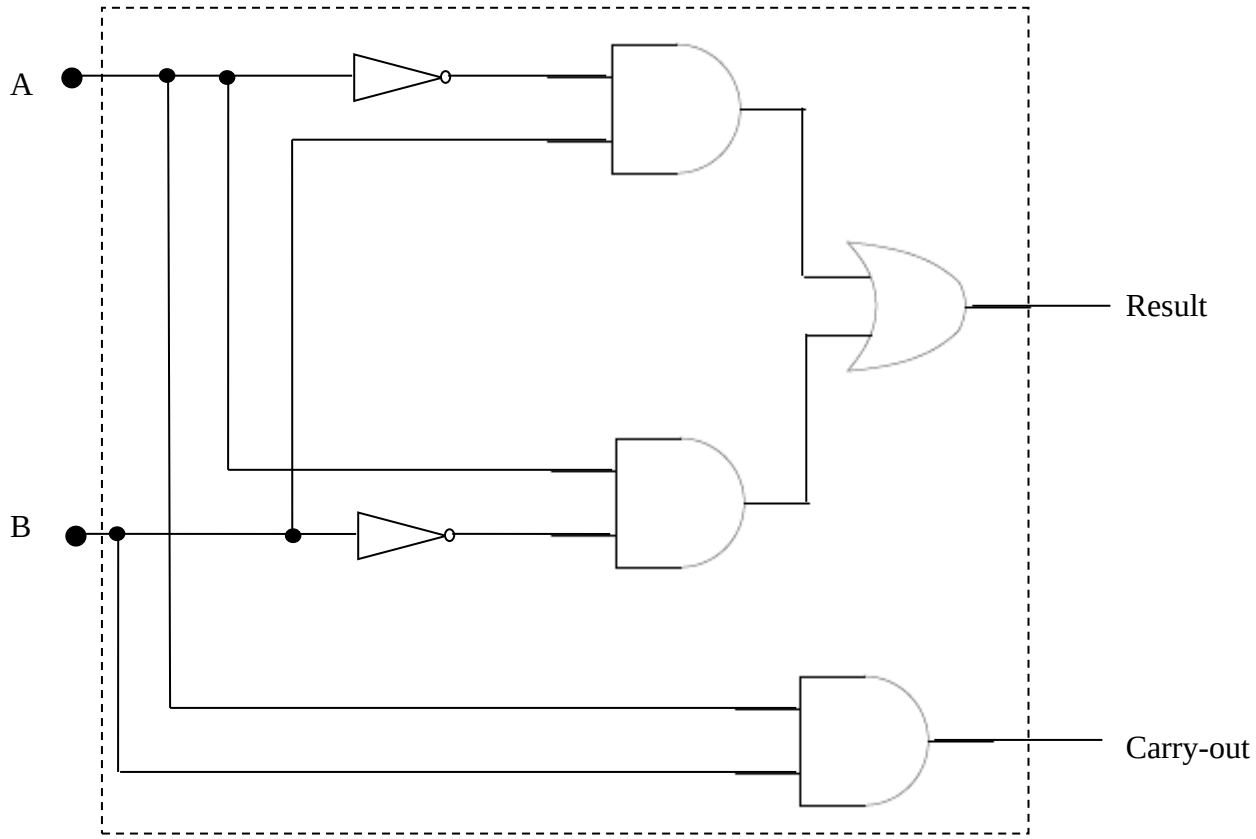
Note how we "count" across the inputs A and B to ensure that we have covered every possibility.

The two outputs are dealt with separately.

So: Result = A ● ~B + ~A ● B

and Carry-out = A ● B

And the circuit(s) look like this. Note that Result and Carry-out each have their own circuit.



This is called a **half-adder**.

The full-adder circuit

We now build a **full-adder** to handle Carry-in as well.

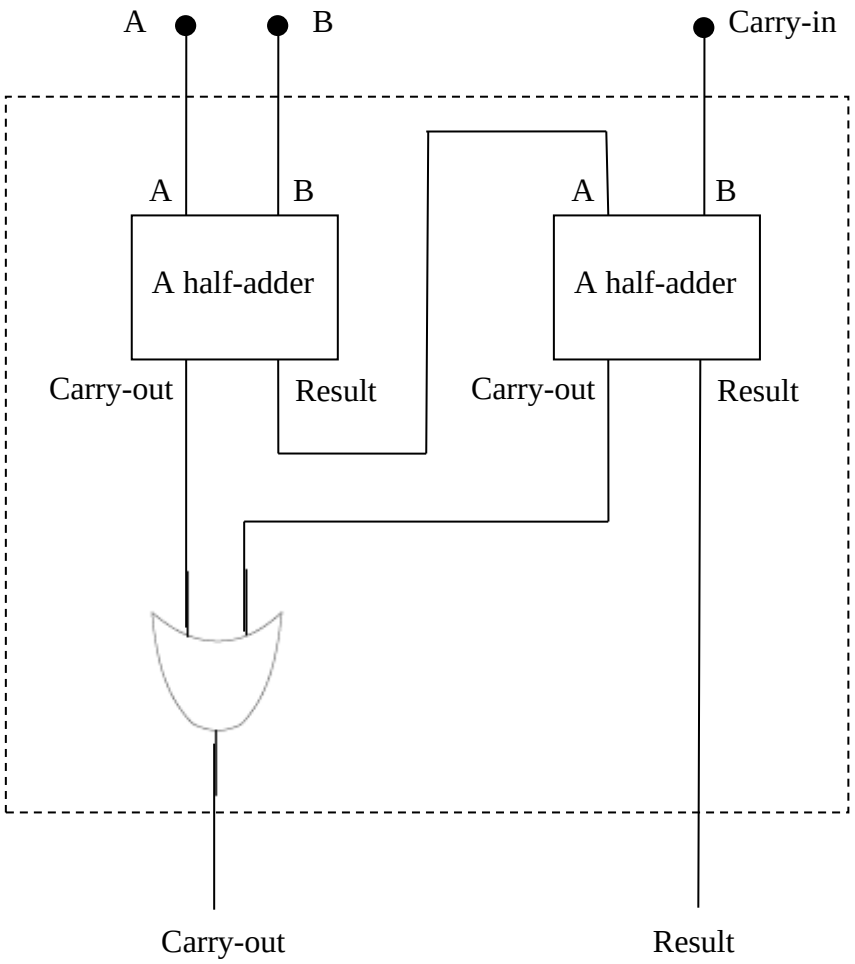
The truth table for a **full-adder**:

A	B	Carry-in	Result	Carry-out
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Note that the Carry-out is the same as the output from a voting circuit!

Now we need to create the Boolean expressions and design the circuit. But wait a moment…

We notice that we are simply performing two additions – first we add A and B and then we add the result to Carry-in so all we have to do is to combine two half-adders as follows:



Does this work? Check it against the truth table!

Using the full-adder:

To **store** a binary number we group flip-flops (bits) together in words.

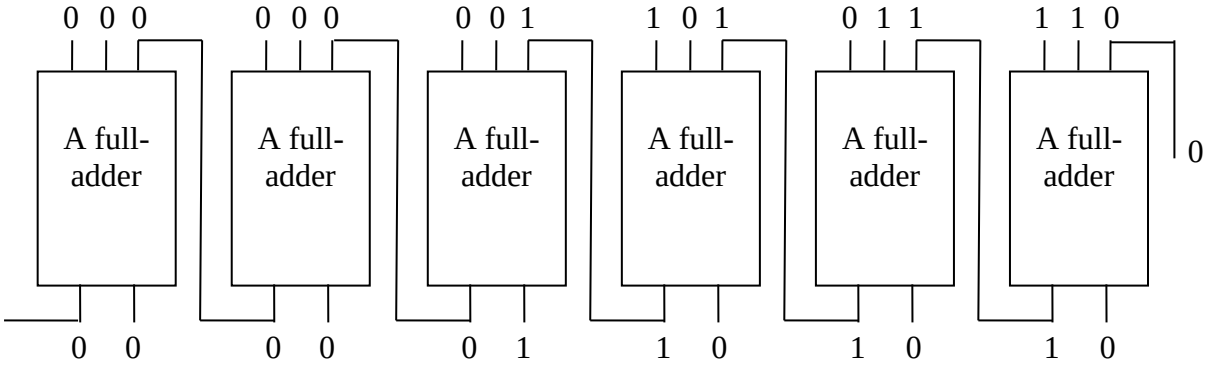
To **add** binary numbers we group adders together – we need one for each bit in the word.

For example: let us assume a 6 bit word and add the numbers 3 and 5

Supply the bits making up 5 down line A for each adder (left-hand input below).

Supply the bits making up 3 down line B for each adder (central input below).

Supply an initial Carry-in of zero. Then add.



The bits coming out of the result lines make up the binary number 00 1000 = 8.

The initial Carry-in line is not always zero. We use it when we want to:

(a) **increment a number** (e.g. x++ in C) by doing the following:
supply the number x down the A lines into each adder.
switch OFF all the B lines, i.e. make all B lines equal to zero.
supply a one on the initial Carry-in line.

(b) **make a negative number** using twos complement by doing the following:
feed the number into the complement circuits (what do these look like?)
supply the result from the complement circuits down the A lines into each adder.
switch OFF all the B lines, i.e. make all B lines equal to zero.
supply a one on the initial Carry-in line.

Representation of Real Numbers

Real numbers (e.g. -25.87) can be divided into four parts: the sign, the integer part, the decimal point and the fraction. The digits in the fraction are sitting in columns: 10^{-1}, 10^{-2}, 10^{-3}, etc. Note that you can keep inserting zeros onto the right-hand side of the fraction without changing the number.

There are various ways of representing real numbers:

A. fixed-point notation

There are a fixed number of decimal places (or the point is fixed in the "window"). For example:

Calculate 0.12 * 0.12 using fixed-point arithmetic with 3 decimal places.

So: 0.120 * 0.120 = 0.014 (always use 3 fixed decimal places)

But this is the wrong answer!! The correct answer is 0.0144. So we decide not to use fixed-point.

B. floating-point notation

The decimal point is free to "float" and we use an **exponent**.

For example, the number 0.000682 can be written as $0.682 * 10^{-3}$

As another example, the number 152.379 can be written as $0.152379 * 10^{3}$

(Note: the convention of writing 0.something is known as "**scientific notation**")

(Historical note: the floating-point system was used by the Kerala School of Mathematics in India in approximately 1350).

A reminder on how to do arithmetic with exponents

Multiplication: $(a * 10^m) * (b * 10^n) = (a * b) * 10^{m+n}$

Example 1: $25 * 4 = 0.25 * 10^2 * 0.4 * 10^1 = 0.1 * 10^3 = 100$

Example 2: $12 * 4 = 0.12 * 10^2 * 0.4 * 10^1 = 0.048 * 10^3 = 48$

Division: $(a * 10^m) / (b * 10^n) = (a / b) * 10^{m-n}$

Example 1: $100 / 25 = 0.1 * 10^3 / 0.25 * 10^2 = 0.4 * 10^1 = 4$

Example 2: $48 / 2 = 0.48 * 10^2 / 0.2 * 10^1 = 2.4 * 10^1 = 24$

Adding/Subtracting: 1. First, unnormalise the smaller number to get the exponents the same.

2. add/subtract the numbers and leave the exponent unchanged.

Example: $1000 + 85 = 0.1 * 10^4 + 0.85 * 10^2$

$= 0.1 * 10^4 + 0.0085 * 10^4 = 0.1085 * 10^4 = 1085$

Floating point numbers in binary

Example: $52 = 11\ 0100 = 1.101 * 2^5$

Notes: We now have base 2 instead of base 10 (and a binary point instead of a decimal point!)

The exponent still tells us how many places the point has "floated"

The digits in the fraction sit in columns: 2^{-1}, 2^{-2}, 2^{-3}, etc

We "normalise" to **1.fraction** rather than **0.something** (as in scientific notation)

Converting Binary real numbers to Decimal real numbers

1. The sign will remain unchanged.

2. Remove the integer part and convert as for integer values.

3. Convert the fraction by using the columns (powers of two).

Example: 11.101 sign = positive

integer part = 11 = 3

fraction = $0.101 = 1 * 2^{-1} + 0 * 2^{-2} + 1 * 2^{-3}$

$= 0.5 + 0.125 = 0.625$

Final result: 11.101 = 3.625

Converting Decimal real numbers to Binary real numbers

1. The sign will remain unchanged.

2. Remove the integer part and convert as for integer values.

3. Convert the fraction by multiplying by 2 as follows:

Example1: -5.375 sign = negative

integer part = 5 = 101

fraction = 0.375 * 2 = **0**.750 (keep track of bold number and ignore it)

0.750 * 2 = **1**.500 (keep track but ignore the 1 in next line)

0.5 * 2 = **1**.0

Final result: -5.375 = -101.011

Example 2: 0.25 sign = positive

integer part = zero

fraction = 0.25 * 2 = **0**.5

0.5 * 2 = **1**.0

Final result: 0.25 = 0.01

How to store real numbers in a computer

First convert the decimal number to a floating-point binary number (see previous lecture).

Then normalise the binary number to the form: 1.fraction * $2^{exponent}$

So we need to store the **sign**, the **fraction** and the **exponent**.

Example 1: -3.25 = 11.01 normalised to 1.101 * 2^{1}

This will be stored as: sign = negative
fraction = 101
exponent = 1

Example 2: 0.125 = 0.001 normalised to 1.0 * 2^{-3}

This will be stored as: sign = positive
fraction = 0
exponent = -3

A problem with exponents:

Exponents can be positive and negative so we need a way to represent this in binary. There are four approaches to representing negative numbers in a computer:

Sign & Magnitude	used by floats
Ones Complement	not used
Twos Complement	used by integers

Excess Notation used by exponents

So we use **excess notation** to represent exponents. This means adding the **excess value** to the exponent before storing the number.

Exponent example 1: exponent is 2, excess value is 15. Thus 17 will be stored.

Exponent example 2: exponent is -4, excess value is 18. Thus 14 will be stored.

By choosing the right excess value we can avoid ever storing a negative number.

A common representation for Real Numbers

There are various approaches to storing real numbers in a computer. These approaches are different on different computers, and change as new computers are produced with a higher number of available bits for storage. A commonly used approach is as follows:

Single precision: 32 bits with 1 bit for the sign, 8 bits for the exponent and 23 bits for the fraction.

Exponents are stored using excess notation with an excess value of 127.

Double precision: 64 bits with 1 bit for the sign, 11 bits for the exponent and 52 bits for the fraction.

Exponents are stored using excess notation with an excess value of 1023.

Example 1: -3.25 would be stored as follows (using single precision and 32 bits):

a) convert -3.25 to binary and normalise the number (see above) to get $1.101 * 2^1$

b) sign = negative (1 in binary)

exponent = 1 + 127 = 128 (1000 0000 in binary)

fraction = 101

c) thus -3.25 would be stored as `1 1000 0000 1010 0000 0000 0000 0000 000`

Example 2: 0.125 would be stored as follows (using single precision and 32 bits):

a) convert 0.125 to binary and normalise the number (see above) to get $1.0 * 2^{-3}$

b) sign = positive (0 in binary)

exponent = -3 + 127 = 124 (0111 1100 in binary)

fraction = 0

c) thus 0.125 would be stored as `0 0111 1100 0000 0000 0000 0000 0000 000`

Converting back from storage to decimal

What decimal number is represented by the following? Assume single precision (32 bits).

`1 1000 0011 1000 1000 0000 0000 0000 000`

sign = negative

exponent = 131 – 127 = 4

fraction = 10001

Thus we have $1.10001 * 2^4 = 11000.1 = -24.5$

Special cases

Zero can never be normalised to the form 1.something so 32 zeros is used to represent zero.

By the way, what should 32 zeros actually represent? Eg. the exponent is 0 – 127 = –127

Infinity is represented as: 0 1111 1111 0000 0000 0000 0000 0000 000

An exponent of all zeros is used only to represent zero and an exponent of all ones is used only to represent infinity.

WARNING!

Long numbers get cut off when placed into a floating point variable. Thus many real numbers are not actually stored correctly in a computer. Small errors can be increased by calculations, particularly multiplication.

Programming bits in C++

C and C++ are two of the very few programming languages that allow us to directly manipulate the binary values stored in the computer. In order to do this, we use low level operations, often known as bitwise operations. In this section we cover the main bitwise operations that are available.

Using hexadecimal in a C++ program

A computer always stores integer values as binary numbers. We use hexadecimal as a shorthand notation for writing binary. One hexadecimal digit = a group of 4 bits. We can work directly with hexadecimal in a C++ program. Remember that any value you store in a computer program is in binary. There are various ways to read a value (e.g. enter a decimal number) and to display a value.

```
#include <iostream>
using namespace std;

int value;
char chr;

int main() {
  cout << "Enter a 2-digit hexadecimal number ";
  cin >> hex >> value;
  chr = value;
  cout << "Value is " << value << endl;
  cout << "Character is " << chr << endl;
  cout << "Hexadecimal value is ";
  cout << hex << uppercase << value << endl;
}
```

Interesting things to notice in this program:

- the value is entered as hexadecimal but displayed as both decimal and hexadecimal

- the value is also copied to a character (i.e. we use the value as an ASCII code)
- `hex` means "display the number in hexadecimal instead of decimal"
- `uppercase` means "use capital letters in your hexadecimal numbers, e.g. AB"

Sometimes we want to load a hexadecimal value into a variable. We use `0x`, for example:

```
#include <iostream>
using namespace std;

int value;

int main() {
  value = 0x41;    // this means load with hexadecimal value 41
  cout << "Value is " << value << endl;
}
```

Hexadecimal is useful to load zero or one into specific bits. For example, if I wanted to store the bit pattern 00110101 in 8 bits, I would do the following:

Step 1: set up a char variable (which has 8 bits)
Step 2: convert the binary number 0011 0110 into hexadecimal (36)
Step 3: write the program:

```
#include <iostream>
using namespace std;

char value;

int main() {
  value = 0x36;    // this means load with hexadecimal value 36
  cout << "Value is " << (int)value << endl;
}
```

(Note how I used `(int)value` to force it to display the character as a value and not as a character.)

Signed and unsigned

Computers use a system to represent (integer) negative numbers that means that one of the bits (the sign bit) is not available for use. For example, if we had a char (8 bits) the largest number that we can store is 0111 1111. The zero is in the sign bit and signifies that we are dealing with a positive number. Note that 0111 1111 is the hexadecimal number 7F and then check what this program displays:

```
#include <iostream>
using namespace std;

char value;

int main() {
  value = 0x7F;    // this means load with hexadecimal value 7F
  cout << "Value is " << (int)value << endl;
}
```

But we have a problem if we put a one in the sign bit (as follows). Note that 1111 1111 is the hexadecimal number FF.

```
#include <iostream>
using namespace std;

char value;

int main() {
  value = 0xFF;    // this means load with hexadecimal value 7F
  cout << "Value is " << (int)value << endl;
}
```

We now get an unexpected result displayed.
We can avoid this problem by telling the system that we want to use "unsigned" variables. In other words, make use of the sign bit as one of the bits in the number (and not as a sign bit). For example:

```
#include <iostream>
using namespace std;

unsigned char value;

int main() {
  value = 0xFF;    // this means load with hexadecimal value 7F
  cout << "Value is " << (int)value << endl;
}
```

If a variable is "unsigned", it can use all the bits BUT it can never store a negative number.

Note: most data types (e.g. char, short, int) can be defined as `unsigned`.

Bitwise Operations

There are three groups of operations in C++:

Arithmetic operations: `*, /, +, -, %`

Logical operations: `>, <, ==, !=, >=, <=, &&, ||, !`

Bitwise operations as follows:

&	AND	|	OR	^	XOR
~	complement	<<	left shift	>>	right shift

How bitwise operations work

& do the AND operation across every corresponding bit in the two numbers.
1 & 1 produces 1 and any other combination produces zero

| do the OR operation across every corresponding bit in the two numbers.
0 | 0 produces 0 and any other combination produces one

^ do the XOR operation across every corresponding bit in the two numbers.
Same as for OR except that 1 ^ 1 produces 0.

Also known as “exclusive or”.

~ do the complement operation on a single numbers.
Switch every zero to a one and every one to a zero.

<< shift every bit in the single number to the left.
E.g. 0101 shift left by 1 position produces 1010
Note that shift left is a super-fast way of multiplying by 2.

>> shift every bit in the single number to the right.
E.g. 1100 shift right by 1 position produces 0110
Note that shift right is a super-fast way of dividing by 2.

Examples (write a program to try these out):

```
int x, y, a;
x = 3;
y = 5;
a = x & y;   // the answer is 1 – see working below
```

Show the working: x = 0011 (only writing the last 4 bits of 32)
y = 0101
AND 0001 (1 & 1 = 1 and all others = 0)

```
a = x | y;   // the answer is 7 – see working below
```

Show the working: x = 0011 (only writing the last 4 bits of 32)
y = 0101
OR 0111 (0 | 0 = 0 and all others = 1)

```
a = x ^ y;   // the answer is 6 – see working below
```

Show the working: x = 0011 (only writing the last 4 bits of 32)
y = 0101
XOR 0110 (0 ^ 0 = 0, 1 ^ 1 = 0 and all others = 1)

Swap two numbers without using temp:

```
int a, b;
a = 5;
b = 3;
a = a ^ b;          a = 0101 ^ 0011 = 0110
b = a ^ b;          b = 0110 ^ 0011 = 0101    final value for b is 5
a = a ^ b;          a = 0110 ^ 0101 = 0011    final value for a is 3
```

Complement example:

```
unsigned char k, m;
k = 33;             // k = 0010 0001
m = ~k;             // m = 1101 1110 = DE16 = 13*16 + 14 = 22210
```

Note that m is not a negative number as we are using unsigned variables.

The twos complement system complements and adds one to find the negative number:

```
nt x;
x = 17;
m = ~x + 1;   // m will now have the value -17
```

Masking

The process of "covering" most of a (number) so that it is not affected by an operation.
(Similar to masking tape which covers part of the wall so it is not affected by paint).

Use AND to check whether a bit is one or zero:

```
int x, mask, result;

int main() {
  x = 2;          // try different numbers here
  mask = 4;
  result = x & mask;
  if (result != 0) {
    cout << "The 3rd bit is set to one\n";
  }
}
```

Everything in the mask is zero except for one bit. When we AND the number with the mask, it will make every bit in the result equal to zero except the bit in the mask.

Use OR to set a bit to one:

```
int x, mask, result;

int main() {
  x = 0x42;  (decimal 66)     // try different numbers here
  mask = 0x20;  (mask with a one in the sixth bit)
  result = x | mask;
  cout << "x is " << (char) x;
  cout << " and result is " << (char) result << endl;
}
```

Set the sixth bit in a character (of 8 bits) to a one (mask = 32) to change a capital letter to a small letter.
Example: `'A' = 65 = 0100 0001.` Now set the sixth bit `= 0110 0001 = 97 = 'a'`

Use XOR to reset a bit to zero:

```
x = 66;   // try different numbers here
mask = 32;
x = x | mask;  // set the bit
x = x ^ mask;  // reset the bit
```

Reset the sixth bit in a character to a zero (mask = 32) to change a small letter to a capital letter.

Shifting

The process of moving bits along the binary number.

```
result = x << 3;
```
This will move the bits 3 places to the left and fill in on the right with zeros.

```
result = x >> 2;
```
This will move the bits 2 places to the right and fill in on the left with zeros.

Shift left example:

```
x = 5;   // x = 0101
result = x << 1;  // result = 1010 (shift and fill the gap with a zero)
```

The result is 10. Note that "shift left" once is the same as multiply by 2. What about shift twice?

Shift right example:

```
x = 6;   // x = 0110
result = x >> 1;  // result = 0011 (shift and fill the gap with a zero)
```

The result is 3. Note that "shift right" once is the same as divide by 2. What about shift twice?

WARNING

You need to be aware that certain bitwise operations (e.g. shift) may change the most significant bit (the sign bit). This may have unexpected consequences in a program. For this reason, many programs only use bitwise operations with **unsigned** variables.

For example, the following code is expected to produce a result of 32800 (= 16400 * 2):

```
short x, result;
x = 16400;
result = x << 1;
```

However, it produces an incorrect result of -32736 because 1 has shifted left into the sign bit. Perform the operation in binary (on paper) to see what is happening.

In order to solve this, we can use **unsigned** variables as follows:

```
unsigned short x, result;
x = 16400;
result = x << 1;
```

Now it produces the correct result of 32800 because the most significant bit is no longer used as a sign bit.